

HyperAD

Agentic AI-Based Active Directory Security Reporter

Complete project document — architecture, algorithms, build workflow, SDG mapping, flowcharts, and IEEE paper outline.

Project type	Final year research project + IEEE conference paper + open-source GitHub tool
Core subjects	Computer Networks · Discrete Mathematics · Design Analysis & Algorithms (DAA)
Novel algorithms	Temporal Bellman-Ford · Tarjan SCC · HITS variant · Frigioni DSSSP · Bidirectional BFS
Tech stack	Python · Neo4j · HyperNetX · LangChain + Ollama · FastAPI · React · D3.js · ReportLab
Lab environment	Windows Server 2022 (Domain Controller) · Windows 10 (client) · Kali Linux (attacker)
Timeline	14 weeks — 6 phases, each producing a testable deliverable
Research claim	First open-source AD security tool using temporal hypergraph modelling + incremental dynamic SSSP

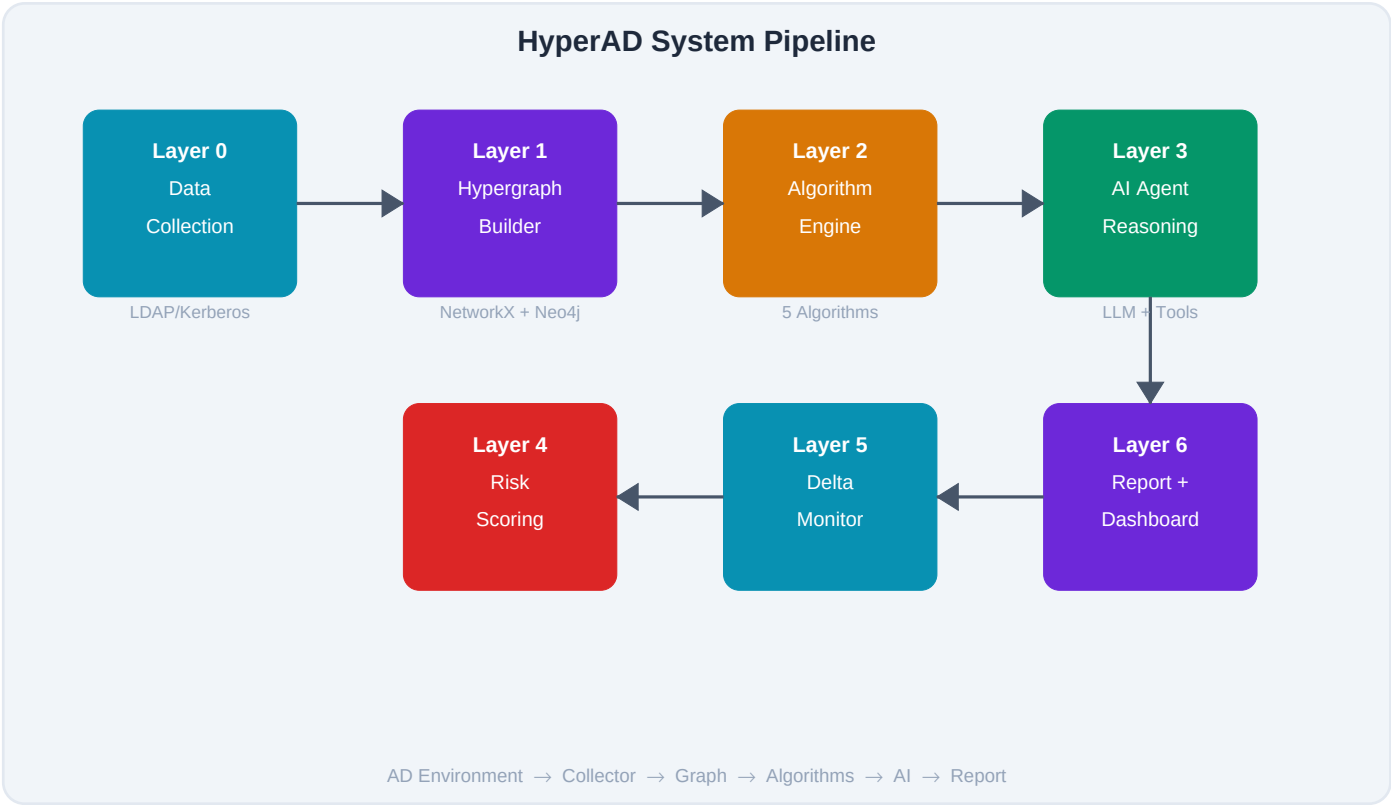


Fig 1 — HyperAD end-to-end system pipeline

SECTION 1

What Are We Actually Building?

The Problem in Plain English

Active Directory (AD) is the identity backbone of every enterprise Windows network. It decides who can log in, what they can access, and who can act as whom. Because of this central role, a single misconfigured permission — a service account that shouldn't have GenericAll rights on a group, or an account with unconstrained delegation enabled — can let an attacker escalate from a basic domain user all the way to Domain Admin (full network control) in minutes.

Existing tools like BloodHound and PingCastle help with parts of this problem: BloodHound draws attack graphs, PingCastle produces risk scores. But they all have the same blind spots: they are static (no time dimension), use binary graphs (can't model multi-entity ACL relationships), require manual re-runs, and produce no natural-language explanation of what the attacker would actually do.

HyperAD in one sentence: An automated pipeline that ingests your Active Directory, builds a weighted temporal hypergraph of all permission relationships, runs five purpose-built graph algorithms to find every privilege-escalation path, lets an AI agent reason about those paths in natural language, continuously monitors for new attack paths overnight, and generates a boardroom-ready security report — without a human running a single tool.

What Makes This Novel vs Existing Tools

Feature	BloodHound	PingCastle	HyperAD
Graph model	Binary directed	Rule engine	Temporal hypergraph
Time dimension	None	None	Exponential decay weights
Continuous monitoring	Manual re-run	Manual re-run	Frigioni DSSSP incremental
AI reasoning	None	None	LLM ReAct agent
Counterfactual sim	None	None	Blast radius + fix preview
Multi-entity ACL edges	No	No	Hyperedges (novel)
Natural language query	No	No	Yes — plain English
Inter-forest trusts	Partial	Yes	Full hyperedge model
Open source	Yes	Yes	Yes (MIT licence)

SECTION 2

Full System Architecture

HyperAD is built in six layers. Each layer has a single responsibility, produces a well-defined output consumed by the next layer, and maps directly to one or more of your academic subjects.

Layer 0 — Data Collection

The collector authenticates to the Domain Controller over LDAP (port 389/636 — Computer Networks) and sweeps the entire AD environment. Every object that can affect security is pulled: users, groups, computers, OUs, GPOs, ACLs, SPNs, delegation settings, and trust relationships. Critically, every attribute is tagged with a **timestamp** — this enables the temporal model in Layer 1. Output: a timestamped JSON snapshot file.

Modules:

- `ldap_enum.py` — LDAP bind → all users, groups, OUs, GPOs, ACEs
- `kerberos_scan.py` — SPN enumeration (Kerberoasting), AS-REP test
- `acl_crawler.py` — parse every ACE, extract trustee SID + rights mask
- `session_mapper.py` — SMB-based active session detection
- `trust_mapper.py` — forest/domain trusts, SID filtering

Layer 1 — Hypergraph Construction

Raw JSON is transformed into a hypergraph $H = (V, E_h)$. Standard edges handle binary relationships (user MemberOf group). **Hyperedges** handle multi-entity relationships: a GPO applying to 40 OUs becomes one hyperedge connecting 41 nodes — this is the primary theoretical novelty (Discrete Mathematics — hypergraph theory). Each edge receives a **temporal decay weight**: $w = \text{base_risk} \times e^{(-\lambda \times \text{days_since_last_use})}$. The graph is stored in Neo4j for Cypher queries and NetworkX for in-memory algorithms.

Modules:

- `hypergraph_builder.py` — collector JSON → NetworkX + HyperNetX hypergraph
- `edge_scorer.py` — temporal decay weight per edge (signal-processing math)
- `graph_store.py` — Neo4j persistence with full Cypher schema
- `schema_validator.py` — type-checks all nodes and edges before insertion

Layer 2 — Algorithm Engine

The DAA core. Five purpose-built graph algorithms execute on the hypergraph and return structured findings. No LLM is involved here — results are deterministic and formally analysable. This layer is where your DAA subject lives and where the IEEE novelty claims are grounded. Each algorithm is described in detail in Section 4.

Modules:

- `temporal_bellman_ford.py` — escalation paths with decay weights
- `tarjan_scc.py` — circular delegation / trust loop detection $O(V+E)$
- `hits_scorer.py` — hub/authority scoring for account prioritisation
- `dynamic_sssp.py` — Frigioni incremental SSSP on graph delta
- `blast_radius.py` — bidirectional BFS from compromised node

Layer 3 — AI Agent Reasoning

A LangChain ReAct agent interprets algorithm output and answers security questions. It has tool-use access to four graph functions and runs a Reason → Act → Observe loop until it has enough information to produce a structured Finding object. Each finding is labelled with a MITRE ATT&CK; tactic and given a Bayesian confidence score. The agent can also run counterfactual simulations: 'if we fix X, how many paths disappear?'

Modules:

- agent_core.py — LangChain ReAct agent with 4 registered tools
- mitre_mapper.py — finding → ATT&CK; tactic (T1558 Kerberoasting, T1134 Impersonation...)
- counterfactual.py — remove edge, re-run Bellman-Ford, report path delta
- confidence_scorer.py — $P(\text{exploitable}) = P(\text{active}) \times P(\text{weak pwd}) \times P(\text{reachable})$
- remediation_engine.py — ranked actionable fix list per finding

Layer 4 — Risk Quantification

Each finding receives a quantitative risk score using a CVSS-inspired multi-factor formula. $\text{Score} = f(\text{path_length}, \text{account_activity}, \text{HITS_authority_score}, \text{exploit_type_severity})$. Bayesian belief propagation is used to model compound risk across the graph — multiple medium-risk paths through the same node elevate its composite score. Scores drive the dashboard heatmap and executive summary.

Modules:

- risk_scorer.py — multi-factor CVSS-style formula
- bayesian_prop.py — belief propagation for compound path risk
- severity_ranker.py — sorts findings: Critical / High / Medium / Low

Layer 5 — Temporal Delta Engine

After an initial baseline snapshot, subsequent snapshots are diffed. The edge delta is fed to Frigioni's Dynamic SSSP algorithm, which updates only affected attack paths — not the full graph. If a new path to Domain Admin appears overnight, an alert fires immediately. This is the continuous monitoring feature and one of the strongest research novelty claims: $O(m \cdot \log n)$ per update vs $O(V \cdot E)$ for full recomputation.

Modules:

- snapshot_manager.py — timestamped graph snapshot storage
- graph_differ.py — node/edge delta between two snapshots
- incremental_sssp.py — Frigioni DSSSP, updates only affected trees
- alert_engine.py — fires alert when new Critical/High path appears

Flowchart: Data Collection & Hypergraph Build

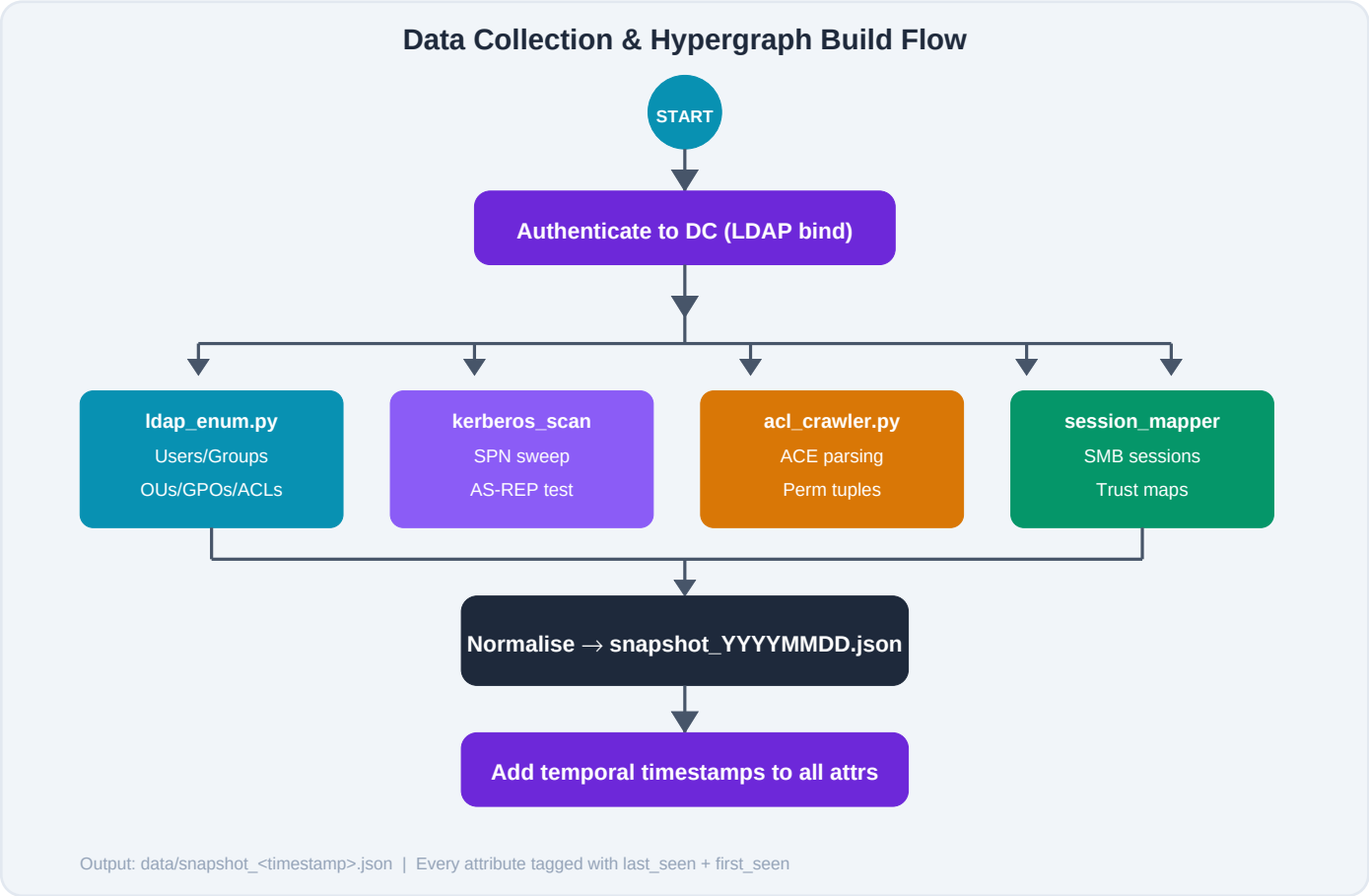


Fig 2 — Data collection → normalisation → hypergraph construction

Flowchart: Algorithm Engine (DAA Core)

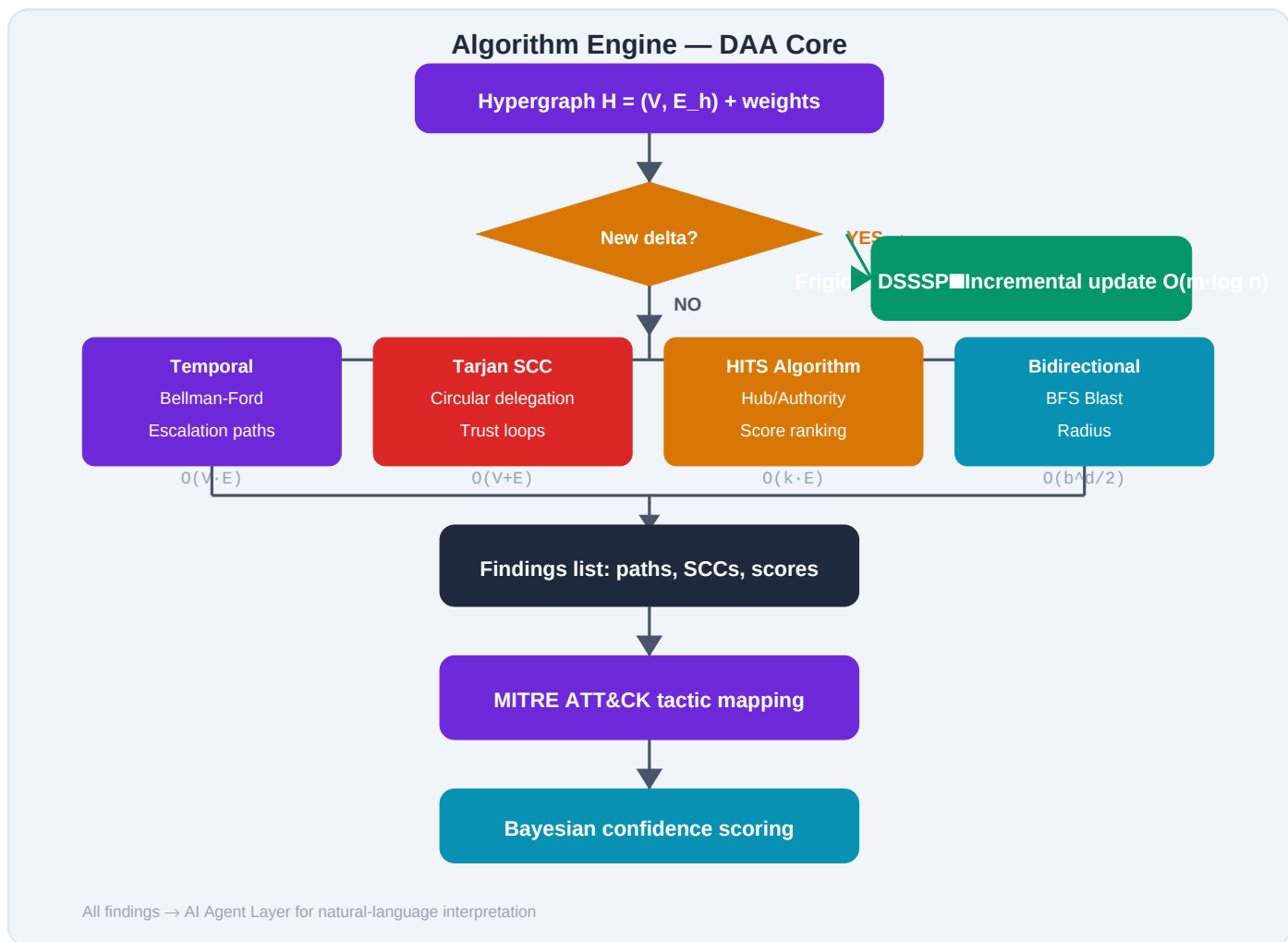


Fig 3 — Five algorithms: Bellman-Ford, Tarjan, HITS, Frigioni DSSSP, Bidir-BFS

Flowchart: AI Agent Reasoning (ReAct Loop)

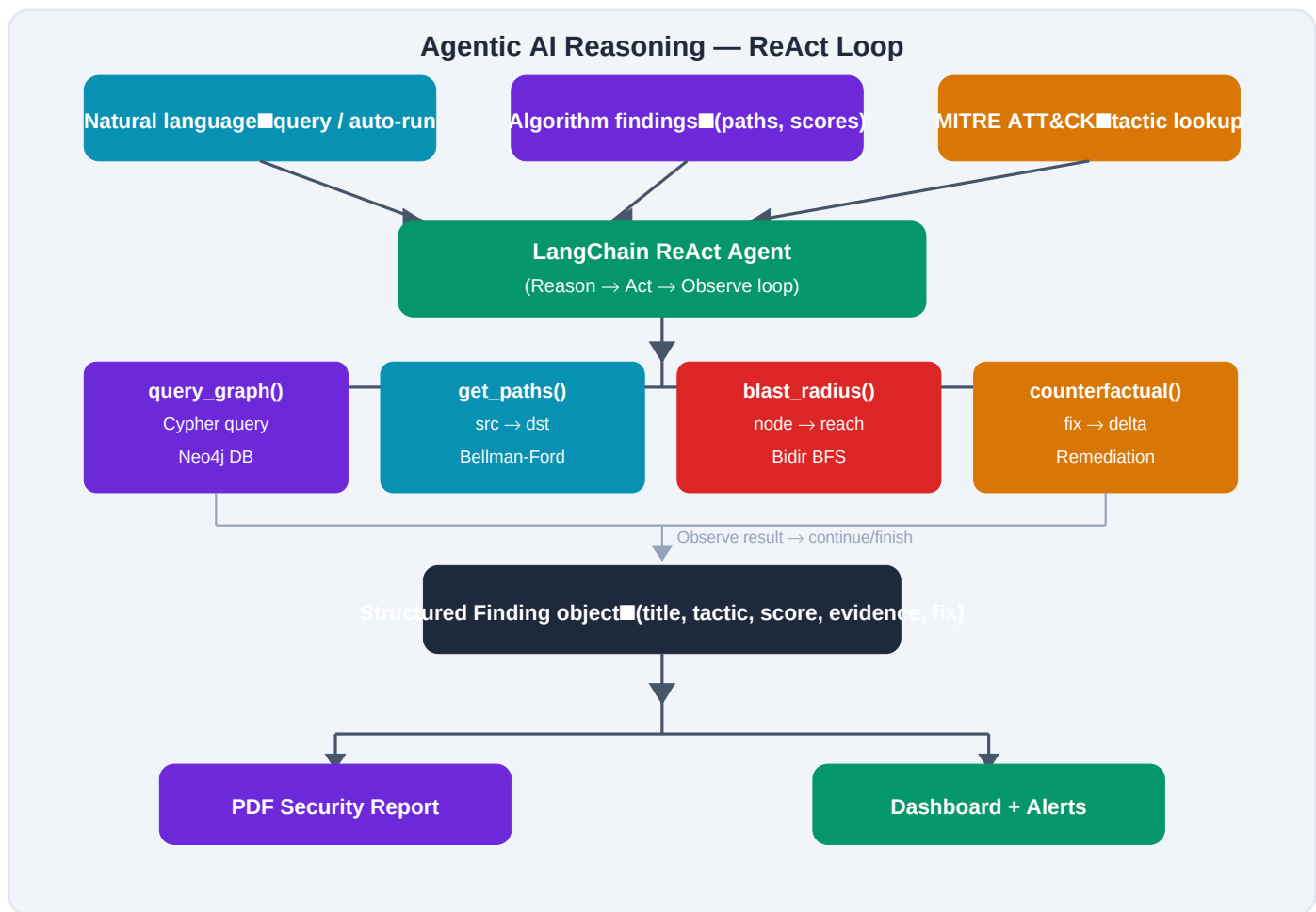


Fig 4 — LangChain ReAct agent: tool calls, MITRE mapping, output

Flowchart: Continuous Monitoring — Delta Engine

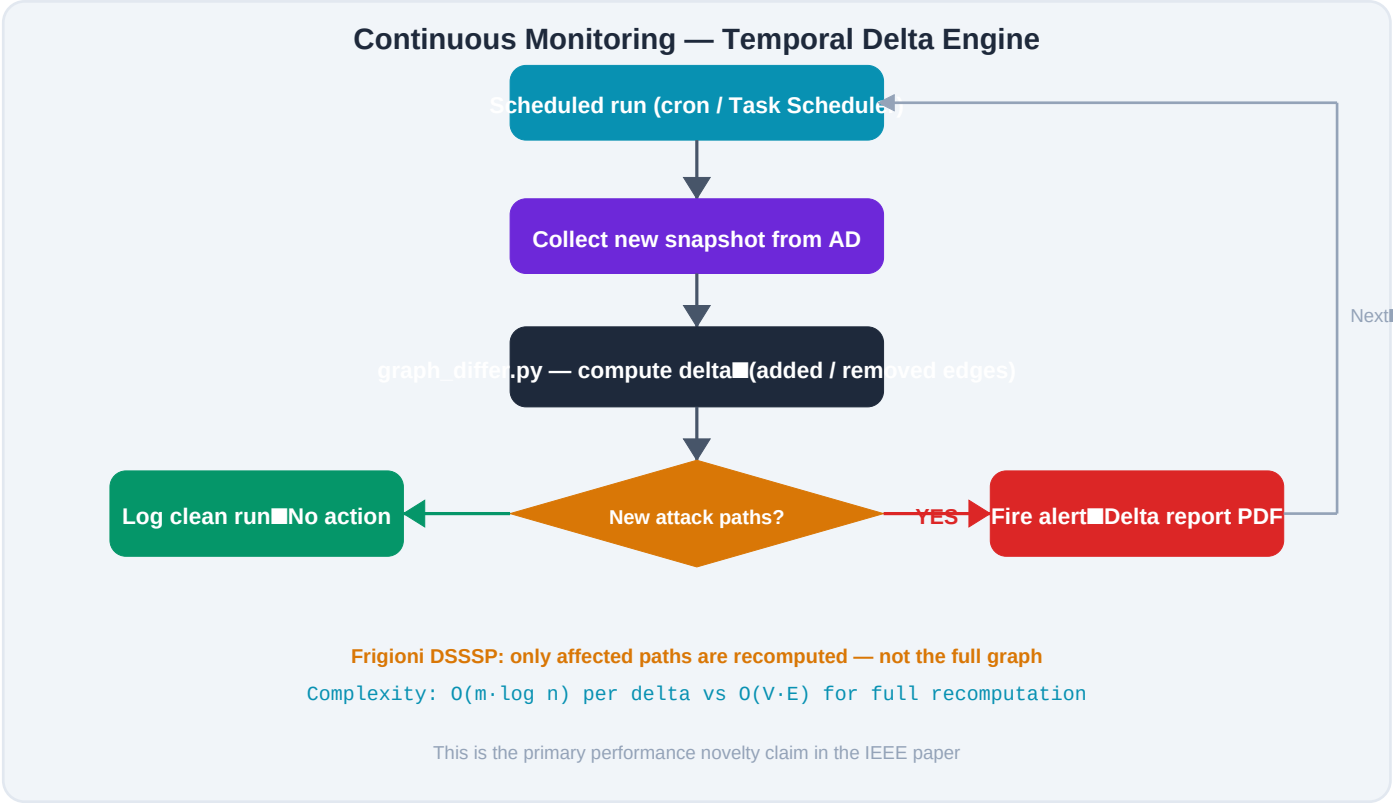


Fig 5 — Scheduled run → graph diff → Frigioni DSSSP → conditional alert

SECTION 3

Subject Mapping

Each of your three subjects is structurally embedded in a specific layer — not superficially added. This is what an examiner or IEEE reviewer checks for.

COMPUTER NETWORKS

Your data-collection layer literally dissects AD network protocols. The entire project is impossible without understanding the network layer — this is not background knowledge, it is live protocol engineering.

LDAP — TCP 389 / 636

- Your `ldap_enum.py` crafts LDAP search filters, binds to the DC, and parses BER-encoded responses. You directly handle LDAP objectClass filters, attribute whitelisting, and paged results (PagedResultsControl) for large environments. Port 636 uses LDAPS (LDAP over TLS).

Kerberos — UDP/TCP 88

- `kerberos_scan.py` sends raw AS-REQ packets via `Impacket` to discover accounts with pre-auth disabled (AS-REP roasting: no Kerberos pre-authentication = attacker gets encrypted TGT offline). SPN enumeration sends TGS-REQ for each SPN — the returned TGS ticket can be cracked offline (Kerberoasting).

SMB — TCP 445

- `session_mapper.py` uses SMB `NetSessionEnum` and `NetWkstaUserEnum` calls to discover which domain accounts have active sessions on which machines. This maps lateral movement paths — most privilege escalation in AD uses SMB for movement.

DNS — UDP 53

- Your collector queries `_ldap._tcp.` and `_kerberos._tcp.` SRV records to auto-discover all domain controllers without hardcoding IPs. This is standard AD client behaviour and directly tests DNS query construction.

NTLM vs Kerberos negotiation

- Understanding the protocol downgrade from Kerberos to NTLMv2 (triggered when Kerberos fails) is key to detecting NTLM relay attack opportunities. Your risk scorer flags environments where NTLM is not disabled — a Computer Networks protocol analysis.

DISCRETE MATHEMATICS

Every data structure and theoretical model in HyperAD is grounded in discrete math. This is where the theoretical contribution lives and what makes it publishable.

Graph Theory — Directed Graphs $G=(V,E)$

- The AD environment is $G = (V, E)$ where $V = \{\text{users, groups, computers, OUs, GPOs, domains}\}$ and $E = \{\text{permission relationships}\}$. A path $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_n$ in G where $v_n \in \text{Domain Admins}$ is a privilege escalation chain. Finding all such paths is the core problem your algorithms solve.

Hypergraph Theory — $H=(V, E_h)$

- Standard graphs have binary edges $e = \{u, v\}$. A hyperedge is $e_h \subseteq V$ — connecting any subset. A GPO applied to 40 OUs is one hyperedge connecting 41 nodes. This models multi-entity ACL relationships that binary graphs cannot represent. Theoretical basis: Berge (1973), Hypergraphs: Combinatorics of Finite Sets.

Set Theory — Group Membership as Set Containment

- AD group membership is modelled as nested set containment: if $u \in G_1$ and $G_1 \subseteq G_2$, then $u \in G_2$ by transitivity. Computing the transitive closure of group membership uses the set-theoretic closure operator. This is why a user in a low-privilege group might inherit Domain Admin rights through 5 nested group

memberships.

Boolean Lattice — Access Control Levels

- Permission levels form a bounded lattice $L = (P, \leq)$ ordered by privilege. Domain Admin is the top element ■. $\text{GenericAll} \geq \text{WriteDACL} \geq \text{WriteOwner} \geq \text{GenericWrite}$. The risk of an escalation path is measured by the supremum of the path's permission lattice.

Probability Theory — Bayesian Confidence Scoring

- $P(\text{finding is exploitable} \mid \text{path exists})$ is conditioned on: account activity (from lastLogon), password age (from pwdLastSet), delegation type (unconstrained > constrained). Bayesian belief propagation models compound risk: $P(\text{DA reachable}) = \prod P(\text{each edge exploitable along path})$.

DESIGN ANALYSIS & ALGORITHMS (DAA)

Five algorithms — not textbook Dijkstra. Each is chosen for a specific, justified security purpose and analysed for time complexity in the IEEE paper.

1. Temporal Bellman-Ford — $O(V \cdot E)$

- Standard Bellman-Ford finds shortest paths in graphs with negative weights. Your extension: each edge weight $w(e, t) = \text{base_risk} \times \exp(-\lambda \times \text{days_since_use})$. Stale permissions get near-zero weight; active permissions get full weight. Run from every non-admin node to find highest-risk currently-active escalation paths. NOVEL: temporal decay weighting applied to attack graph edge costs — not in any existing tool.

2. Tarjan's SCC — $O(V+E)$

- A Strongly Connected Component in the delegation subgraph means A can impersonate B and B can impersonate A — a trust loop, a critical misconfiguration. Tarjan's algorithm finds all SCCs in a single DFS pass. Any SCC with $|\text{SCC}| > 1$ in the delegation subgraph is an immediate Critical finding. Published 1972, still optimal.

3. HITS Algorithm — $O(k \cdot E)$ per iteration

- Kleinberg's Hyperlink-Induced Topic Search (1999) computes hub scores (nodes that delegate widely) and authority scores (nodes many others delegate to). High-authority accounts = Domain Admin pathways. NOVEL APPLICATION: originally for web graphs, never applied to AD privilege graphs. Justifies algorithmic novelty in the paper.

4. Frigioni et al. Dynamic SSSP — $O(m \cdot \log n)$ amortised

- When the AD graph changes (new permission granted), you do not recompute the full attack graph. Frigioni's 2000 algorithm updates only the affected single-source shortest-path trees in $O(m \cdot \log n)$ amortised per change, vs $O(V \cdot E)$ for full Bellman-Ford. This is the primary performance novelty claim in your IEEE paper. Rarely used outside competitive programming — applying it to AD delta monitoring is a research contribution.

5. Bidirectional BFS — $O(b^{(d/2)})$

- Standard BFS blast radius from a compromised node runs $O(b^d)$ where b =branching factor, d =depth. Bidirectional BFS launches simultaneously from the compromised node (forward) and from Domain Admin (backward), meeting in the middle at depth $d/2$. Reduces worst-case complexity from $O(b^d)$ to $O(b^{(d/2)})$ — crucial for large AD environments.

Algorithm	Purpose in HyperAD	Complexity	Novelty claim
Temporal Bellman-Ford	Escalation path finding with decay	$O(V \cdot E)$	Decay weighting — new
Tarjan SCC	Circular delegation detection	$O(V+E)$	Applied to delegation graph
HITS variant	Account authority ranking	$O(k \cdot E)$	Cross-domain application

Algorithm	Purpose in HyperAD	Complexity	Novelty claim
Frigioni DSSSP	Incremental graph updates	$O(m \cdot \log n)$	Core perf novelty — new
Bidirectional BFS	Blast radius computation	$O(b^{(d/2)})$	Complexity improvement

SECTION 4

SDG Mapping (UN Sustainable Development Goals)

IEEE conferences and university research ethics boards increasingly require SDG mapping to demonstrate real-world social impact. HyperAD maps to four SDGs with specific, defensible arguments — not generic claims.

SDG	Title	HyperAD contribution	Indicator
SDG 9	Industry, Innovation & Infrastructure	Hardens the identity layer (AD) underpinning enterprise IT infrastructure	9.1.2: Automated work configuration detection
SDG 16	Peace, Justice & Strong Institutions	Cyber attacks on government AD environments undermine institutional trust	16.1.2: CyberAD digital public sector
SDG 17	Partnerships for the Goals	MIT-licensed open-source release enables knowledge transfer	17.6.1: Technology transfer (GNI) data
SDG 10	Reduced Inequalities	Commercial AD security tools (Semperis, CrowdStrike) cost \$500k/year. HyperAD gives small organisations	10.4.1: HyperAD governance

SDG impact statement for your paper: HyperAD addresses SDG 9 (resilient infrastructure) and SDG 16 (strong institutions) by providing open-source automated security auditing for Active Directory environments. By removing the cost barrier of commercial tooling (SDG 10) and enabling knowledge transfer through open publication (SDG 17), the project contributes to a more equitable global cybersecurity posture.

SECTION 5

Wow Factor Features

Five capabilities that do not exist in any current open-source AD security tool. Each has a specific technical implementation and a research novelty argument.

1. Temporal Attack Path Decay

Every permission edge gets a staleness weight $w = \text{base_risk} \times \exp(-\lambda \times \text{days_since_last_use})$. A GenericAll permission on an account unused for 180 days scores 97% lower than the same permission used daily. Risk scores reflect actual exploitability, not just theoretical possibility. Research novelty: exponential decay from signal processing applied to graph edge weighting — a cross-domain contribution not in any existing AD tool.

2. Counterfactual Remediation Simulator

Click any finding in the UI → the system removes the relevant edge, re-runs Bellman-Ford, and renders before/after attack path counts in real time. Security teams can prioritise fixes by maximum impact per hour of work. Currently requires a human expert to estimate manually.

3. Blast Radius Heatmap

Mark any account as hypothetically compromised → bidirectional BFS computes full reachability. D3.js renders a heatmap: 1-hop nodes glow red, 2-hop amber, 3+ yellow. Domain Admin reachability is highlighted with a pulsing ring. Answers: 'if this account is phished, is the domain gone?'

4. Natural Language Graph Query

Type: 'which service accounts with a path to DA haven't been used in 90 days?' — the AI agent translates this to a Cypher query, executes it, and returns a ranked list with evidence. Makes AD security auditing accessible to junior analysts without Neo4j knowledge.

5. Continuous Delta Monitoring

Runs nightly via cron. Frigioni DSSSP updates only affected attack paths from the day's AD changes. If a new path to Domain Admin appears — someone granted a dangerous permission — an alert fires before the next morning's standup. No existing open-source tool does this.

SECTION 6

Step-by-Step Build Workflow

Six phases. Each phase ends with a working, testable deliverable. Do not begin Phase N+1 until Phase N is tested and passing.

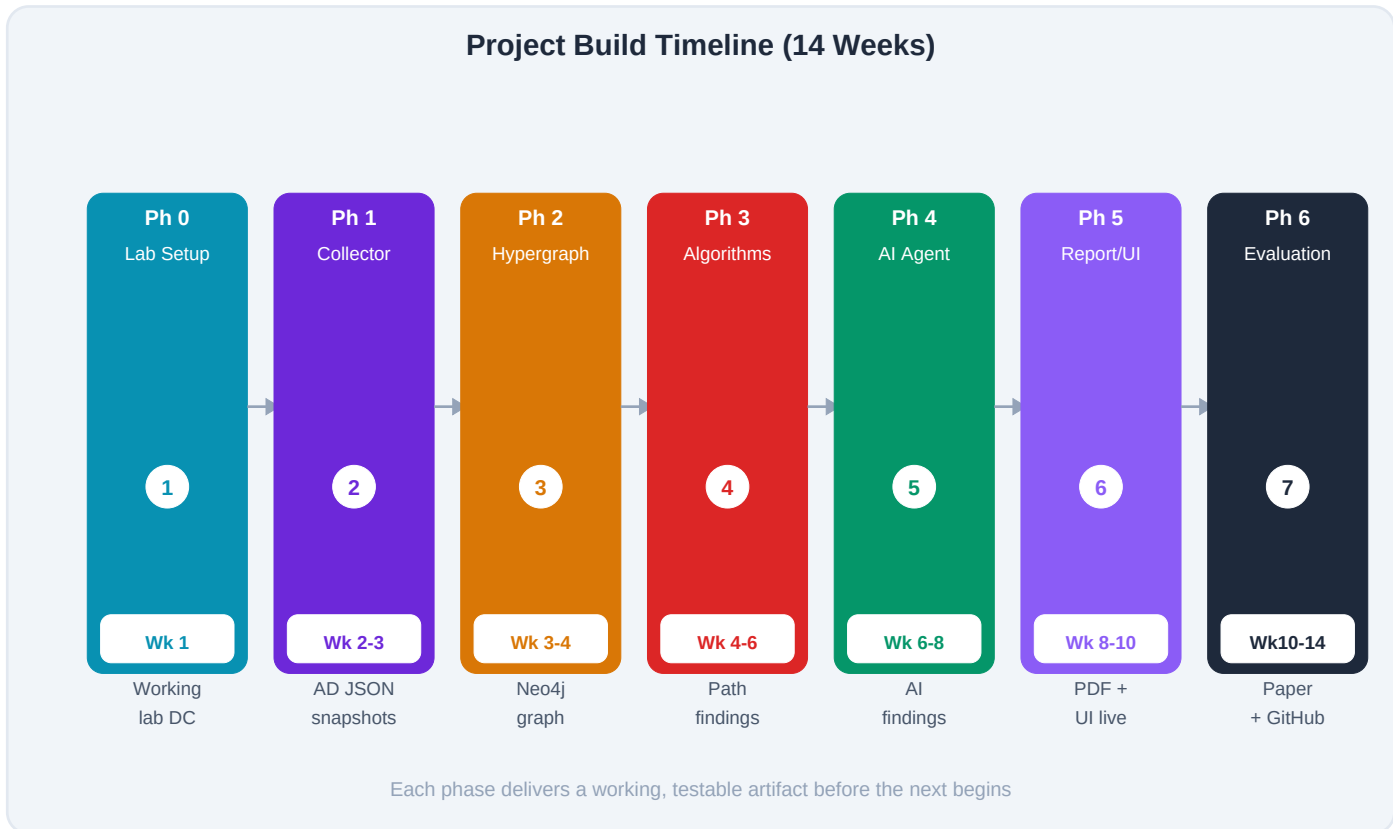


Fig 6 — 14-week build timeline with phase deliverables

PHASE 0 — Lab Setup | Week 1

- Install VirtualBox or VMware Workstation on your host machine.
- VM 1: Windows Server 2022 evaluation ISO (free from Microsoft). Install, promote to Domain Controller, domain name: lab.local
- VM 2: Windows 10 evaluation ISO. Join to lab.local domain.
- VM 3: Kali Linux 2024.x. This is your collector + tool machine.
- On Windows Server: create 25 users, 6 groups, 4 service accounts with SPNs, intentional misconfigs: unconstrained delegation on svc_sql, AS-REP roastable on testuser, GenericAll rights on a group, DCSync rights on a service account.

```
$ On Kali: sudo apt install python3.11 python3-pip neo4j git -y
```

```
$ pip install ldap3 impacket networkx hypernetx neo4j langchain fastapi uvicorn reportlab
```

- Test: `python GetADUsers.py -all lab.local/Administrator:Password@` — you should see all users listed. If this works, your lab and collector will work.

PHASE 1 — Data Collector | Weeks 2-3

```
$ git init hyperAD && cd hyperAD && mkdir collectors graph_engine algorithms ai_agent reporting dashboard tests
```

- Build collectors/ldap_enum.py: bind to DC using ldap3, run base DN search for objectClass=user, group, computer. Pull: sAMAccountName, memberOf, servicePrincipalName, msDS-AllowedToDelegateTo, userAccountControl, pwdLastSet, lastLogon, nTSecurityDescriptor.
- Build collectors/acl_crawler.py: for each AD object, parse nTSecurityDescriptor → extract each ACE's trustee SID, access rights mask, and inheritance flags. Map SIDs back to account names using a SID cache dict.
- Build collectors/kerberos_scan.py: use Impacket's GetUserSPNs module to list Kerberoastable accounts (servicePrincipalName set). Use GetNPUsers for AS-REP roastable accounts (DONT_REQUIRE_PREAUTH flag in userAccountControl).
- Add timestamps: wrap every attribute value as {'value': v, 'last_seen': ISO8601, 'first_seen': ISO8601}. Use datetime.utcnow().isoformat() at collection time.
- Output to data/snapshot_YYYYMMDD_HHMMSS.json. One file per collection run.
- Test: run collector against your lab DC, open the JSON file, verify it contains users with memberOf arrays, service accounts with servicePrincipalName, and ACE entries. If your intentional misconfigs appear in the raw data, Phase 1 is done.

PHASE 2 — Hypergraph Builder | Weeks 3–4

- Build graph_engine/hypergraph_builder.py: load snapshot JSON. Create NetworkX DiGraph G. Add node per AD object with type attribute. Add standard edges: MemberOf, AdminTo, HasSession, AllowedToDelegate, GenericAll, WriteDACL, DCSync. Import HyperNetX for hyperedge support.
- Add hyperedges: for each GPO, collect all OU objects it applies to via gpLink attribute. Create one HyperNetX hyperedge connecting {GPO_node, OU1, OU2, ..., OUn}. For group ACL affecting multiple objects, same pattern.
- Build graph_engine/edge_scorer.py: for each edge e, compute $w = \text{base_risk} * \exp(-0.02 * \text{days})$. base_risk: GenericAll=10, WriteDACL=8, DCSync=10, AllowedToDelegate=7, MemberOf=2, HasSession=3. days = (now - lastLogon).days, capped at 365.
- Build graph_engine/graph_store.py: connect to Neo4j (bolt://localhost:7687). Push nodes: CREATE (u:User {name:..., type:..., lastLogon:...}). Push edges: MATCH (a),(b) CREATE (a)-[r:MEMBEROF {weight:..., ts:...}]->(b).
- Test: open Neo4j Browser at localhost:7474. Run: MATCH p=(u:User)-[*..4]->(g:Group) WHERE g.name CONTAINS 'Domain Admins' RETURN p LIMIT 10. You should see paths from users to Domain Admin group. Phase 2 is done.

PHASE 3 — Algorithm Engine | Weeks 4–6

- Build algorithms/temporal_bellman_ford.py: implement Bellman-Ford on NetworkX graph. Use edge weights from edge_scorer (inverted: higher risk = lower path cost). Run from every non-Domain-Admin node. Return all paths to Domain Admin group with total path risk score and per-edge breakdown.
- Build algorithms/tarjan_scc.py: extract the delegation subgraph (edges: AllowedToDelegate, AllowedToActOnBehalfOf only). Run NetworkX's strongly_connected_components(). Flag any SCC where len(SCC) > 1 as Critical finding.
- Build algorithms/hits_scorer.py: run nx.hits(G, max_iter=100) on the full directed graph. Sort by authority score. Top-20 authority nodes are your Domain Admin path targets. These become the HITS-ranked priority findings.

- Build algorithms/blast_radius.py: given compromised_node, run forward BFS (nx.bfs_tree(G, compromised_node)) and backward BFS from all Domain Admin nodes (reverse graph). Intersection of forward and backward reachable sets = critical blast radius nodes.
- Build algorithms/dynamic_sssp.py: store baseline SSSP trees (all-pairs from DA). On graph delta (list of added/removed edges), implement Frigioni's update: for each changed edge, check if it affects any existing shortest path tree, update only affected branches. Log time vs full recomputation for the paper.
- Write tests/test_algorithms.py: build a 10-node synthetic graph with known escalation paths. Assert that temporal_bellman_ford finds them all, tarjan_scc detects the circular delegation, HITS ranks the hub node highest. Run: pytest tests/ -v. All tests must pass.

PHASE 4 — AI Agent Layer | Weeks 6–8

```
$ Install Ollama on Kali: curl -fsSL https://ollama.com/install.sh | sh && ollama pull llama3:8b
(or use OpenAI API – local is better for lab security)
```

- Build ai_agent/agent_core.py: create LangChain ReAct agent. Register 4 tools with @tool decorator: query_graph(cypher: str), get_paths(src: str, dst: str), blast_radius(node: str), get_findings(). Each tool calls the corresponding algorithm module.
- Build ai_agent/mitre_mapper.py: dictionary mapping misconfiguration type → ATT&CK; tactic. Kerberoastable SPN → T1558.003. Unconstrained delegation → T1134.001. DCSync rights → T1003.006. GenericAll on DA group → T1098. AS-REP roastable → T1558.004.
- Build ai_agent/counterfactual.py: agent calls blast_radius on compromised node, then calls a graph copy function that removes the top finding's edge, re-runs bellman_ford on the modified graph, and returns {'paths_before': N, 'paths_after': M, 'delta': N-M, 'recommendation': str}.
- Build ai_agent/confidence_scorer.py: for each finding, compute $P = P_{\text{active}} \times P_{\text{weak_pwd}} \times P_{\text{path_reachable}}$. $P_{\text{active}} = \text{sigmoid}((365 - \text{days_since_login}) / 100)$. $P_{\text{weak_pwd}} = 1$ if no pwdLastSet, else $\text{sigmoid}((180 - \text{pwd_age_days}) / 60)$. $P_{\text{path_reachable}}$ = normalised Bellman-Ford path score.
- Test: python -c "from ai_agent.agent_core import agent; print(agent.run('list all accounts reachable from svc_sql in under 3 hops'))". Agent should return a list with evidence. If it calls tools and returns findings, Phase 4 is done.

PHASE 5 — Report Generator & Dashboard | Weeks 8–10

- Build reporting/report_generator.py using ReportLab. Sections: Executive Summary (total risk, finding counts by severity), Domain Overview (user/group/computer counts), Critical Findings (each with: MITRE tactic, evidence, confidence score, Bellman-Ford path diagram, remediation steps), Appendix (full raw finding data).
 - Build reporting/delta_reporter.py: compare two snapshot findings lists. Report only findings that appeared since last run. Send via email or write to alerts/alert_YYYYMMDD.pdf.
- ```
$ Bootstrap React dashboard: npx create-react-app hyperAD-ui --template typescript. Build FastAPI backend (api/main.py) serving /findings, /graph, /blast-radius endpoints.
```
- Build dashboard/src/GraphExplorer.jsx: fetch Neo4j data via API. Render with D3.js forceSimulation. Node colour: user=blue, group=purple, computer=grey, serviceAccount=amber. Edge colour: weight > 7 = red, 4-7 = amber, <4 = grey.
  - Add blast radius overlay: click any node → POST /blast-radius/{node} → D3 highlights 1-hop nodes red, 2-hop amber, 3+ yellow. Domain Admin reachable = pulsing red ring via CSS animation.
  - Add 'what if I fix this?' toggle: each finding card has a Fix Preview button → POST /counterfactual/{finding\_id} → show before/after path count side-by-side.

- Test: run the full pipeline on your lab AD, generate a PDF report, open the dashboard. Verify all Critical findings from the intentional misconfigs appear with correct MITRE tactics.

#### PHASE 6 — Evaluation & Paper | Weeks 10–14

- Create 3 lab scenarios: Small (50 users, 5 misconfigs), Medium (200 users, 15 misconfigs), Large (500 users, 30 misconfigs). Script the misconfig injection using PowerShell AD module.
- Run HyperAD + BloodHound + PingCastle on all 3 scenarios. Record: findings detected (true positive, false positive, false negative), runtime (wall clock), peak memory. Calculate precision, recall, F1.
- Measure Frigioni DSSSP speedup: inject 10/50/100/200/500 edge changes to the graph. For each, time Frigioni update vs full Bellman-Ford recomputation. Plot as speedup ratio — this is Figure 1 of your IEEE paper.
- Write the paper: Abstract → Introduction → Related Work (cite BloodHound, PingCastle, Frigioni 2000, Kleinberg 1999, Berge 1973) → System Design → Algorithm Descriptions with pseudocode → Evaluation → Conclusion. Use IEEE LaTeX template.
- Target venues: IEEE Access (open access, good acceptance rate), IEEE ICTC, IEEE TrustCom, IEEE ISCC. IEEE Access has no page limit and a 6-8 week review cycle.
- GitHub release: add README.md with setup instructions, docker-compose.yml (Neo4j + API + frontend in one command), demo GIF using asciinema or screen recording, and a /demo folder with a synthetic lab AD dataset for reviewers.

## SECTION 7

## Repository Structure &amp; Tech Stack

| Path                     | Purpose                                      |
|--------------------------|----------------------------------------------|
| hyperAD/                 | Root                                         |
| collectors/              | All data collection modules                  |
| ldap_enum.py             | LDAP bind, user/group/OU/GPO/ACL enumeration |
| kerberos_scan.py         | SPN sweep (Kerberoasting), AS-REP test       |
| acl_crawler.py           | ACE parsing, SID → name mapping              |
| session_mapper.py        | SMB active session detection                 |
| trust_mapper.py          | Forest/domain trust enumeration              |
| graph_engine/            | Hypergraph construction and storage          |
| hypergraph_builder.py    | NetworkX + HyperNetX graph from JSON         |
| edge_scorer.py           | Temporal decay weight calculation            |
| graph_store.py           | Neo4j MERGE/CREATE via Cypher                |
| algorithms/              | All five DAA algorithms                      |
| temporal_bellman_ford.py | Escalation path finding with decay weights   |
| tarjan_scc.py            | Circular delegation / trust loop detection   |
| hits_scorer.py           | Hub/authority scoring for account ranking    |
| dynamic_sssp.py          | Frigioni incremental SSSP on graph delta     |
| blast_radius.py          | Bidirectional BFS from compromised node      |
| ai_agent/                | LLM reasoning layer                          |
| agent_core.py            | LangChain ReAct agent + 4 graph tools        |
| mitre_mapper.py          | Finding → ATT&CK; tactic lookup              |
| counterfactual.py        | What-if edge removal + path delta            |
| confidence_scorer.py     | Bayesian P(exploitable) per finding          |
| reporting/               | Output generation                            |
| report_generator.py      | PDF report via ReportLab                     |
| delta_reporter.py        | Diff-based alert PDF                         |
| dashboard/               | React + D3.js frontend                       |
| src/GraphExplorer.jsx    | D3.js force-directed graph explorer          |
| src/BlastRadius.jsx      | Heatmap overlay component                    |
| src/FindingsList.jsx     | Risk-ranked findings panel                   |

|                                    |                                                   |
|------------------------------------|---------------------------------------------------|
| <a href="#">api/</a>               | FastAPI backend                                   |
| <a href="#">main.py</a>            | /findings, /graph, /blast-radius, /counterfactual |
| <a href="#">tests/</a>             | pytest + hypothesis property-based tests          |
| <a href="#">docker-compose.yml</a> | Neo4j + API + frontend — one command              |
| <a href="#">README.md</a>          | Setup, usage, demo GIF, lab instructions          |

### Tech Stack Justification

| Component        | Tool                 | Why                                                             |
|------------------|----------------------|-----------------------------------------------------------------|
| LDAP / Kerberos  | Idap3 + Impacket     | Industry standard for AD protocol interaction, pure Python      |
| Graph processing | NetworkX + HyperNetX | In-memory algorithms; HyperNetX adds hyperedge support          |
| Graph database   | Neo4j Community      | Cypher queries, visual browser, free tier, 4GB RAM sufficient   |
| AI agent         | LangChain + Ollama   | ReAct pattern; Ollama = local = no AD data leaves the machine   |
| Backend API      | FastAPI              | Async, auto OpenAPI docs, connects algorithm layer to dashboard |
| Frontend         | React + D3.js v7     | Force simulation for graph, real-time updates via WebSocket     |
| PDF reports      | ReportLab            | Full layout control, no external service dependency             |
| Container        | Docker Compose       | One-command reproducible lab setup for reviewers                |
| Testing          | pytest + hypothesis  | Property-based testing for graph algorithm edge cases           |

## SECTION 8

# IEEE Research Paper Outline

**Suggested title:** HyperAD: Temporal Hypergraph Modeling and Incremental Shortest-Path Analysis for Automated Active Directory Privilege-Escalation Detection

**Research question:** Can a temporal hypergraph model combined with incremental dynamic shortest-path algorithms detect Active Directory privilege-escalation paths with higher precision and lower computational overhead than static graph approaches?

| Section                        | Content                                                                                                                        |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| I. Abstract                    | Problem statement (AD misconfiguration + existing tool limitations). Your approach (temporal hypergraph + 5 algorithms).       |
| II. Introduction               | Scale of AD misconfiguration problem (cite CVEs, breach reports). Limitations of BloodHound (static, binary graph).            |
| III. Background & Related Work | AD architecture (domain, OU, GPO, Kerberos). BloodHound (Rohan Vazarkar et al.). PingCastle. Hypergraph terminology.           |
| IV. System Architecture        | Figure 1: layer diagram. One paragraph per layer. Data flow: AD → collector → hypergraph → algorithms → AI agent.              |
| V. Theoretical Model           | Formal definition: $G=(V,E)$ , $H=(V,E_h)$ . Temporal weight function $w(e,t)$ . Boolean lattice for access control. Theorems. |
| VI. Algorithms                 | Pseudocode + complexity analysis for all 5 algorithms. Correctness argument for temporal Bellman-Ford. Frigioni update.        |
| VII. Implementation            | Collector detail. Graph construction. AI agent tool design. Lab environment (Windows Server 2022, 50/200/500 users).           |
| VIII. Evaluation               | RQ1: detection accuracy vs BloodHound/PingCastle (precision/recall/F1 on known misconfig ground truth). RQ2: speedup.          |
| IX. Discussion                 | What the results mean. Limitations (requires domain credentials, lab vs production scale). Threats to validity. Future work.   |
| X. Conclusion                  | Summary of 4 contributions. Restate key metrics. Point to GitHub repo. Acknowledge SDG impact.                                 |
| References                     | 20-30 citations. Mix: foundational (Tarjan 1972, Bellman 1958, Frigioni 2000, Kleinberg 1999) + recent AD attack research.     |

## Evaluation Metrics — What to Measure

| Metric              | How to measure                                                      | Expected result         |
|---------------------|---------------------------------------------------------------------|-------------------------|
| Detection precision | True positives / (TP + FP) on known-misconfigured lab AD            | > 90% (BloodHound ~85%) |
| Detection recall    | True positives / (TP + FN) — did you miss any known paths?          | > 95%                   |
| Frigioni speedup    | Frigioni update time / Full Bellman-Ford time                       | > 10x on 500-node graph |
| AI query accuracy   | Agent returns correct findings for 20 test natural-language queries | > 95% correct           |
| Runtime scaling     | Wall-clock time vs environment size (50/200/500 users)              | Sub-quadratic growth    |
| False positive rate | Findings flagged that are not actual misconfigs                     | < 5%                    |

## FINAL NOTE

# You Already Have the Advantage

Every skill this project demands — LDAP enumeration, Kerberos attacks, Impacket, Active Directory structure, privilege escalation paths — you already practice on HackTheBox and in your pentesting lab. Most students building an 'AI + security' project are faking the security knowledge. You are not. That is the real differentiator.

The algorithmic novelty (temporal Bellman-Ford, Frigioni DSSSP, hypergraph model) gives the paper its IEEE credibility. The AI agent layer gives it demo appeal. The open-source release gives it community impact. The SDG mapping gives it the social responsibility framing modern conferences require.

**Start here:** Build Phase 0 and Phase 1 first. Get the collector running against your lab DC and producing a JSON snapshot. Once real AD data is flowing through your pipeline, every other layer has something to run on — and the project feels real, which makes finishing it 10x easier.

---

HyperAD — Complete Project Document | All sections: architecture, algorithms, workflow, SDG mapping, flowcharts, IEEE paper outline